

10-Day Interview Preparation Guide

Gen AI Engineer | ML Engineer | MLOps Engineer

Joemon Jesudas

Day 1: Python & SQL Fundamentals for AI/ML

Day 2: Machine Learning Fundamentals

Day 3: Advanced ML - XGBoost, Gradient Boosting & Feature Engineering

Day 4: NLP & Text Processing for AI Applications

Day 5: Generative AI, LLMs & Prompt Engineering

Day 6: RAG Architecture & Vector Databases

Day 7: LangChain & LLM Application Development

Day 8: Azure AI Services

Day 9: MLOps - Docker, Kubernetes, CI/CD & FastAPI

Day 10: System Design, Model Monitoring & Mock Interview

DAY 1: PYTHON & SQL FUNDAMENTALS FOR AI/ML

Python Core Concepts

Q: What are Python decorators and how are they used in ML/AI?

A decorator is a function that takes another function as input and extends its behavior without modifying it. In ML/AI, decorators are commonly used for:

- Logging function execution time (useful for profiling model training)
- Caching expensive computations (@lru_cache for repeated predictions)
- FastAPI route decorators (@app.post("/predict")) for serving models
- Retry logic for API calls to Azure OpenAI (handling rate limits)

```
from functools import lru_cache
import time

def timer(func):
    def wrapper(*args, **kwargs):
        start = time.time()
        result = func(*args, **kwargs)
        print(f"{func.__name__} took {time.time()-start:.2f}s")
        return result
    return wrapper

@timer
def train_model(X, y):
    model.fit(X, y)
    return model
```

Q: Explain List Comprehension vs Generator Expression. When to use which?

List comprehension creates the entire list in memory at once. Generator expression yields one item at a time, saving memory.

```
# List comprehension - stores all in memory
embeddings = [get_embedding(chunk) for chunk in chunks]

# Generator - memory efficient for large datasets
embeddings = (get_embedding(chunk) for chunk in chunks)

# Use list when: you need random access, small data, reuse multiple times
# Use generator when: large data, iterate once, streaming processing
```

Interview Tip: In interviews, mention generators when discussing large-scale data processing in ML pipelines.

Q: What is the difference between deepcopy and shallow copy?

Shallow copy creates a new object but references the same nested objects. Deep copy creates completely independent copies of all nested objects. In ML, this matters when copying model parameters, dataframes, or configuration dictionaries.

```
import copy
config = {"model": "xgboost", "params": {"lr": 0.01, "depth": 5}}

shallow = copy.copy(config)      # params dict is shared
deep = copy.deepcopy(config)    # params dict is independent
```

```
# Modifying shallow["params"]["lr"] changes original too!
# Modifying deep["params"]["lr"] does NOT affect original
```

Q: Explain `*args` and `**kwargs` with a practical ML example.

`*args` collects positional arguments as a tuple. `**kwargs` collects keyword arguments as a dictionary. Very common in ML for flexible model configuration.

```
def create_model(model_type, *layers, **hyperparams):
    print(f"Model: {model_type}")
    print(f"Layers: {layers}")
    print(f"Config: {hyperparams}")

create_model("neural_net", 128, 64, 32,
             learning_rate=0.001, dropout=0.2)
# Model: neural_net
# Layers: (128, 64, 32)
# Config: {"learning_rate": 0.001, "dropout": 0.2}
```

Q: What are Python context managers and how are they used in ML?

Context managers handle setup and cleanup automatically using the "with" statement. Essential in ML for managing file handles, database connections, model loading, and GPU memory.

```
# File handling for data loading
with open("data.csv", "r") as f:
    data = f.read()

# Database connection for PGVector
with psycopg2.connect(conn_string) as conn:
    cursor = conn.cursor()
    cursor.execute("SELECT * FROM embeddings")

# Custom context manager for model inference
from contextlib import contextmanager

@contextmanager
def model_inference_mode(model):
    model.eval()    # setup
    yield model
    model.train()   # cleanup
```

SQL for Data Engineering & ML

Q: Write a SQL query to find duplicate records in a claims dataset.

```
SELECT claim_id, policy_number, claim_date, COUNT(*) as cnt
FROM claims
GROUP BY claim_id, policy_number, claim_date
HAVING COUNT(*) > 1
ORDER BY cnt DESC;
```

Q: Explain Window Functions with an ML use case.

Window functions perform calculations across rows related to the current row without collapsing them. Critical for feature engineering in ML.

```
-- Rolling average claim amount per customer (feature engineering)
SELECT
    customer_id,
```

```

claim_date,
claim_amount,
AVG(claim_amount) OVER (
    PARTITION BY customer_id
    ORDER BY claim_date
    ROWS BETWEEN 3 PRECEDING AND CURRENT ROW
) AS rolling_avg_claim,
ROW_NUMBER() OVER (
    PARTITION BY customer_id
    ORDER BY claim_date DESC
) AS claim_recency_rank
FROM claims;

```

Q: How do you write a query for PGVector similarity search?

```

-- Create the vector extension
CREATE EXTENSION vector;

-- Create table with vector column
CREATE TABLE documents (
    id SERIAL PRIMARY KEY,
    content TEXT,
    embedding vector(1536) -- OpenAI ada-002 dimension
);

-- Create index for fast similarity search
CREATE INDEX ON documents
USING ivfflat (embedding vector_cosine_ops)
WITH (lists = 100);

-- Similarity search query
SELECT id, content,
       1 - (embedding <=> query_embedding) AS similarity
FROM documents
ORDER BY embedding <=> query_embedding -- <=> is cosine distance
LIMIT 5;

```

Q: Explain EXPLAIN ANALYZE and query optimization.

EXPLAIN ANALYZE runs the query and shows the actual execution plan with timing. Use it to identify slow queries in your ML data pipelines.

```

EXPLAIN ANALYZE
SELECT * FROM claims
WHERE claim_date > '2024-01-01'
AND claim_type = 'accident';

-- Key things to look for:
-- Seq Scan vs Index Scan (Index is faster)
-- Nested Loop vs Hash Join (Hash is better for large tables)
-- Rows estimated vs actual (big diff = stale statistics)
-- Fix: CREATE INDEX idx_claims_date_type
--      ON claims(claim_date, claim_type);

```

Day 1 Practice Tasks

- Write a decorator that retries a function 3 times with exponential backoff (useful for Azure OpenAI API calls)
- Write a generator that reads a large CSV file in chunks and yields processed batches

- Write SQL to create a feature engineering pipeline: rolling averages, lag features, rank features
- Practice PGVector queries: insert embeddings, similarity search, filtered search

DAY 2: MACHINE LEARNING FUNDAMENTALS

Core ML Concepts

Q: Explain the Bias-Variance Tradeoff.

Bias is the error from wrong assumptions in the model (underfitting). Variance is the error from sensitivity to training data fluctuations (overfitting). The goal is to find the sweet spot.

- High Bias: Model is too simple. Example: Linear regression on non-linear data. Training AND test error are high.
- High Variance: Model is too complex. Example: Deep decision tree. Training error is low but test error is high.
- Solutions for high bias: More features, complex model, polynomial features, reduce regularization
- Solutions for high variance: More training data, regularization (L1/L2), pruning, dropout, cross-validation

Interview Tip: Relate this to your insurance claims project: "We started with linear regression (high bias) and moved to XGBoost which gave us the right bias-variance balance."

Q: Explain Cross-Validation. Why not just use train-test split?

Simple train-test split can give misleading results if the split is unlucky. K-Fold Cross-Validation splits data into K folds, trains K times using a different fold as validation each time, and averages the results.

- K-Fold CV: Most common, K=5 or K=10
- Stratified K-Fold: Preserves class distribution in each fold. Essential for imbalanced data like fraud detection
- Time-Series CV: For temporal data, train on past, validate on future. No random shuffling!

```
from sklearn.model_selection import StratifiedKFold

skf = StratifiedKFold(n_splits=5, shuffle=True, random_state=42)
scores = []
for train_idx, val_idx in skf.split(X, y):
    X_train, X_val = X[train_idx], X[val_idx]
    y_train, y_val = y[train_idx], y[val_idx]
    model.fit(X_train, y_train)
    scores.append(model.score(X_val, y_val))
print(f"Mean CV Score: {np.mean(scores):.4f} +/- {np.std(scores):.4f}")
```

Q: What are L1 and L2 Regularization? When to use which?

Regularization adds a penalty to the loss function to prevent overfitting.

- L1 (Lasso): Adds absolute value of weights as penalty. Drives some weights to exactly zero. Use for feature selection - when you have many features and want to find the important ones.
- L2 (Ridge): Adds squared weights as penalty. Shrinks all weights but none to zero. Use when all features might be relevant but you want to reduce their impact.
- Elastic Net: Combination of L1 and L2. Use when you want both feature selection and weight shrinkage.

```
# L1 - Loss = MSE + lambda * sum(|w|)
# L2 - Loss = MSE + lambda * sum(w^2)

from sklearn.linear_model import Lasso, Ridge, ElasticNet

lasso = Lasso(alpha=0.1)      # L1: some coefficients become 0
ridge = Ridge(alpha=0.1)     # L2: all coefficients shrink
elastic = ElasticNet(alpha=0.1, l1_ratio=0.5) # Mix
```

Q: Explain the confusion matrix and key metrics.

For a binary classifier (e.g., fraudulent claim: Yes/No):

- True Positive (TP): Predicted fraud, actually fraud
- False Positive (FP): Predicted fraud, actually legitimate (Type I error)
- False Negative (FN): Predicted legitimate, actually fraud (Type II error)
- True Negative (TN): Predicted legitimate, actually legitimate
- Accuracy = $(TP+TN)/(TP+TN+FP+FN)$ -- misleading for imbalanced data!
- Precision = $TP/(TP+FP)$ -- "Of all predicted fraud, how many were actually fraud?"
- Recall = $TP/(TP+FN)$ -- "Of all actual fraud, how many did we catch?"
- F1 Score = $2*(Precision*Recall)/(Precision+Recall)$ -- harmonic mean
- AUC-ROC: Area under ROC curve. Measures model ability to distinguish classes across all thresholds.

Interview Tip: For insurance claims: "We prioritized recall because missing a fraudulent claim (FN) is costlier than investigating a legitimate one (FP)."

Q: How do you handle imbalanced datasets?

In insurance, fraud cases or high-liability claims are rare (1-5%). Standard models will predict the majority class.

- Oversampling: SMOTE (Synthetic Minority Oversampling) - creates synthetic minority samples
- Undersampling: Randomly remove majority class samples. Risk: losing information
- Class Weights: Set `class_weight="balanced"` in sklearn models. Penalizes misclassifying minority more.
- Ensemble methods: `BalancedRandomForest`, `EasyEnsemble`
- Threshold tuning: Lower prediction threshold for minority class
- Evaluation: Use F1, Precision-Recall AUC instead of accuracy

```
from imblearn.over_sampling import SMOTE
from sklearn.ensemble import RandomForestClassifier

smote = SMOTE(random_state=42)
X_resampled, y_resampled = smote.fit_resample(X_train, y_train)

# OR use class weights
model = RandomForestClassifier(class_weight="balanced")
model.fit(X_train, y_train)
```

Supervised Learning Algorithms

Q: Explain Decision Trees and how they split.

Decision trees recursively split data based on feature thresholds that maximize information gain (classification) or minimize variance (regression).

- Gini Impurity: Measures how often a randomly chosen element would be misclassified. Gini=0 means pure node. Used in CART.
- Entropy/Information Gain: Measures disorder. Split that reduces entropy most is chosen. Used in ID3, C4.5.
- Pros: Interpretable, no scaling needed, handles mixed data types
- Cons: Overfits easily (high variance), unstable (small data change = different tree)
- Fix overfitting: `max_depth`, `min_samples_split`, `min_samples_leaf`, pruning

Q: Explain Random Forest.

Random Forest is an ensemble of decision trees using bagging (Bootstrap Aggregating).

- Each tree trained on a random bootstrap sample of the data (sampling with replacement)
- At each split, only a random subset of features is considered (reduces correlation between trees)
- Final prediction: majority vote (classification) or average (regression)
- Reduces variance compared to single decision tree while maintaining low bias
- Key hyperparameters: `n_estimators` (100-500), `max_depth`, `min_samples_split`, `max_features`

```
from sklearn.ensemble import RandomForestClassifier

rf = RandomForestClassifier(
    n_estimators=200,
    max_depth=10,
    min_samples_split=5,
    max_features="sqrt", # sqrt of total features at each split
    class_weight="balanced",
    random_state=42,
    n_jobs=-1 # parallel training
)
rf.fit(X_train, y_train)

# Feature importance
importances = rf.feature_importances_
```

Day 2 Practice Tasks

- Implement a complete ML pipeline: load data, EDA, feature engineering, train, evaluate
- Practice explaining bias-variance tradeoff with your insurance claims example
- Calculate precision, recall, F1 by hand for a sample confusion matrix
- Implement SMOTE and compare results with/without it

DAY 3: ADVANCED ML - XGBOOST, GRADIENT BOOSTING

Gradient Boosting & XGBoost (Deep Dive)

Q: Explain Gradient Boosting step by step.

Gradient Boosting builds trees sequentially. Each new tree corrects the errors of the previous ensemble.

- Step 1: Start with a simple prediction (e.g., mean of target for regression)
- Step 2: Calculate residuals (errors) = actual - predicted
- Step 3: Train a new tree to predict the RESIDUALS, not the original target
- Step 4: Add new tree prediction (scaled by learning rate) to ensemble
- Step 5: Repeat steps 2-4 for $n_{\text{estimators}}$ iterations
- Final prediction = initial prediction + $\text{lr} \times \text{tree1}$ + $\text{lr} \times \text{tree2}$ + ... + $\text{lr} \times \text{treeN}$

Why "Gradient"? Because residuals are the negative gradient of the loss function. We are doing gradient descent in function space.

Q: How does XGBoost differ from standard Gradient Boosting?

- Regularization: L1 and L2 regularization on leaf weights (prevents overfitting)
- Sparsity Aware: Handles missing values natively - learns optimal direction for missing values at each split
- Parallel Processing: Parallelizes at the feature level (not tree level) - finds best split faster
- Column Subsampling: Like Random Forest, uses random feature subsets per tree/level
- Weighted Quantile Sketch: Approximates best split points for large datasets
- Cache Optimization: Out-of-core computing, cache-aware access patterns
- Tree Pruning: Uses `max_depth` and prunes backward (unlike GB which stops growing)

Q: Explain key XGBoost hyperparameters and how to tune them.

```
import xgboost as xgb

params = {
    # Most Important
    "learning_rate": 0.1,      # Step size. Lower = more trees needed
    "n_estimators": 500,      # Number of trees. Use early stopping
    "max_depth": 6,           # Tree depth. 3-10. Higher = more complex

    # Regularization
    "reg_alpha": 0,           # L1 regularization (feature selection)
    "reg_lambda": 1,          # L2 regularization (weight shrinkage)
    "min_child_weight": 1,     # Min samples in leaf. Higher = conservative

    # Randomization (reduce overfitting)
    "subsample": 0.8,         # Row sampling per tree
    "colsample_bytree": 0.8,   # Feature sampling per tree

    # For imbalanced data
    "scale_pos_weight": 10,    # ratio of neg/pos samples
}

model = xgb.XGBClassifier(**params)
model.fit(X_train, y_train,
          eval_set=[(X_val, y_val)],
          verbose=50)
```

Interview Tip: In your interview, say: "We used Bayesian optimization with Optuna for hyperparameter tuning, which is more efficient than grid search."

Q: What is early stopping and why is it important?

Early stopping monitors validation performance during training and stops when it stops improving. Prevents overfitting and saves training time.

```
model = xgb.XGBClassifier(
    n_estimators=1000, # set high
    early_stopping_rounds=50, # stop if no improvement in 50 rounds
    learning_rate=0.05
)
model.fit(X_train, y_train,
          eval_set=[(X_val, y_val)],
          verbose=False)
print(f"Best iteration: {model.best_iteration}")
```

Feature Engineering for Insurance Claims

Q: What feature engineering techniques did you use for the claims prediction model?

- Text Features from accident descriptions: TF-IDF vectors, word count, average word length, sentiment score, presence of specific keywords (e.g., "collision", "injury", "hospital")
- Temporal Features: Day of week, month, quarter, time since policy inception, time since last claim, claim frequency in last 12 months
- Aggregation Features: Average claim amount per customer, total claims per policy type, claim count in same region
- Categorical Encoding: One-hot for low cardinality (claim_type: 5 categories), target encoding for high cardinality (agent_id: 1000+ values)
- Interaction Features: policy_age * claim_amount, region + claim_type combinations
- Missing Value Indicators: Binary flag for whether a field was missing (missingness can be informative)

Q: What is SHAP and how did you use it?

SHAP (SHapley Additive exPlanations) explains individual predictions by calculating each feature contribution. Based on game theory.

- Global importance: Which features matter most across all predictions
- Local explanation: Why did the model predict THIS specific claim amount
- Used for: Model debugging, stakeholder communication, regulatory compliance in insurance

```
import shap

explainer = shap.TreeExplainer(xgb_model)
shap_values = explainer.shap_values(X_test)

# Summary plot - global feature importance
shap.summary_plot(shap_values, X_test)

# Single prediction explanation
shap.force_plot(explainer.expected_value,
                shap_values[0], X_test.iloc[0])
```

Day 3 Practice Tasks

- Implement XGBoost with hyperparameter tuning using Optuna
- Practice explaining Gradient Boosting step-by-step on a whiteboard

- Create 5 different feature engineering techniques for a sample dataset
- Generate and interpret SHAP plots for a trained model

DAY 4: NLP & TEXT PROCESSING FOR AI APPLICATION

Text Preprocessing

Q: Explain the text preprocessing pipeline for NLP tasks.

- Tokenization: Splitting text into words/subwords. Word-level (split by space), subword (BPE, WordPiece), character-level
- Lowercasing: "Claim" and "claim" treated the same
- Stop word removal: Remove "the", "is", "at" etc. (but careful - sometimes they matter)
- Stemming vs Lemmatization: Stemming chops word endings (running->runn). Lemmatization uses dictionary (running->run). Lemmatization is more accurate.
- Special character removal, number handling, contraction expansion

Interview Tip: For your interview: "For the liability prediction system, we preprocessed accident descriptions using tokenization, lemmatization, and TF-IDF before feeding into the model."

Q: Explain TF-IDF and how you used it.

TF-IDF (Term Frequency - Inverse Document Frequency) measures word importance in a document relative to a corpus.

- TF = (count of term in document) / (total terms in document)
- IDF = $\log(\text{total documents} / \text{documents containing term})$
- TF-IDF = TF * IDF. High score = term is frequent in this doc but rare across corpus

```
from sklearn.feature_extraction.text import TfidfVectorizer

tfidf = TfidfVectorizer(
    max_features=5000,
    ngram_range=(1, 2), # unigrams + bigrams
    min_df=2,           # ignore very rare terms
    max_df=0.95         # ignore very common terms
)
features = tfidf.fit_transform(accident_descriptions)
# Result: sparse matrix (n_docs, 5000)
```

Word Embeddings & Transformers

Q: Explain the progression: Bag of Words -> Word2Vec -> Transformers.

- Bag of Words / TF-IDF: Simple word counts. No word order or semantics. Sparse, high-dimensional.
- Word2Vec (2013): Dense vectors (100-300 dims). Captures semantics (king-man+woman=queen). But one vector per word regardless of context.
- Transformers / BERT (2018+): Context-dependent embeddings. "Bank" has different embeddings in "river bank" vs "bank account". Uses self-attention mechanism.
- GPT / LLMs: Decoder-only transformers. Pre-trained on massive data. Used for generation, not just embeddings.

Q: What is the Transformer architecture? Explain self-attention.

Transformer uses self-attention to weigh the importance of each word relative to every other word in the sequence.

- Input: Token embeddings + positional encodings (since no recurrence)
- Self-Attention: For each token, compute Query (Q), Key (K), Value (V) vectors. $\text{Attention} = \frac{\text{softmax}(QK^T / \sqrt{d_k})}{\sqrt{d_k}} * V$
- Multi-Head Attention: Multiple attention heads capture different relationships (syntax, semantics, coreference)

- Encoder: Processes input bidirectionally (used in BERT, embeddings)
- Decoder: Generates output autoregressively (used in GPT, text generation)
- Encoder-Decoder: Used in translation, summarization (T5, original Transformer)

Q: What are OpenAI embeddings and how do they work?

OpenAI embedding models (text-embedding-ada-002, text-embedding-3-small) convert text to dense vectors.

- Input: Any text (up to 8191 tokens for ada-002)
- Output: Dense vector (1536 dimensions for ada-002, 256-3072 for text-embedding-3)
- Properties: Similar texts have similar vectors (high cosine similarity)
- Used for: Semantic search, RAG retrieval, clustering, classification

```
from openai import AzureOpenAI

client = AzureOpenAI(
    api_key="your-key",
    azure_endpoint="https://your-resource.openai.azure.com/",
    api_version="2024-02-01"
)

response = client.embeddings.create(
    input="insurance claim for car accident",
    model="text-embedding-ada-002"
)

embedding = response.data[0].embedding # list of 1536 floats
```

Day 4 Practice Tasks

- Implement a text preprocessing pipeline for accident descriptions
- Compare TF-IDF features vs sentence embeddings for text classification
- Explain transformer self-attention mechanism with a diagram
- Write code to generate and store OpenAI embeddings in PGVector

DAY 5: GENERATIVE AI, LLMS & PROMPT ENGINEERING

LLM Fundamentals

Q: What is a Large Language Model (LLM)?

An LLM is a neural network (typically transformer-based) trained on massive text data to predict the next token. Key characteristics:

- Pre-training: Trained on internet-scale text data (books, web pages, code) using self-supervised learning (predict next word)
- Scale: Billions of parameters (GPT-4: rumored 1.7T, Claude: undisclosed, LLaMA-3: 8B-405B)
- Emergence: At scale, models show abilities not explicitly trained for (reasoning, code generation, translation)
- Fine-tuning: Can be adapted to specific tasks with smaller labeled datasets

Q: Explain the difference between GPT (decoder-only) and BERT (encoder-only).

- BERT (Encoder): Reads text bidirectionally. Good for understanding tasks - classification, NER, embedding generation. Uses [MASK] token prediction during training.
- GPT (Decoder): Reads text left-to-right. Good for generation tasks - text completion, chat, code generation. Uses next-token prediction during training.
- T5 (Encoder-Decoder): Best for sequence-to-sequence tasks - translation, summarization.
- For RAG: Embeddings typically use encoder models. Response generation uses decoder models (GPT-4).

Q: What is the temperature parameter in LLMs?

Temperature controls randomness in token selection during generation.

- Temperature = 0: Always picks the most probable token. Deterministic, factual, repetitive.
- Temperature = 0.7: Balanced creativity and coherence. Good for general tasks.
- Temperature = 1.0+: More random, creative, but may hallucinate more.
- For your RAG system: "We used temperature=0 for document extraction (factual) and 0.3 for liability recommendations (slightly creative reasoning)."

Q: What are tokens? How does tokenization work in LLMs?

Tokens are the basic units LLMs process. Not always full words.

- Common tokenizer: BPE (Byte Pair Encoding). "unhappiness" -> ["un", "happiness"] or ["un", "happ", "iness"]
- 1 token is roughly 4 characters or 0.75 words in English
- Context window: Maximum tokens model can process at once. GPT-4: 128K, Claude: 200K
- Cost is per token (input + output). Prompt optimization reduces cost.

Prompt Engineering (Critical for Interviews)

Q: What are the key prompt engineering techniques?

1. Zero-Shot Prompting

Ask the model directly without examples.

```
Classify this insurance claim as HIGH or LOW risk:
"Vehicle collision at intersection, minor injuries reported"
Risk Level:
```

2. Few-Shot Prompting

Provide examples before the actual task.

Classify insurance claims:

Claim: "Rear-end collision, no injuries" -> LOW

Claim: "Multi-vehicle pileup, hospitalization" -> HIGH

Claim: "Windshield crack from debris" -> LOW

Claim: "Head-on collision, multiple injuries" ->

3. Chain-of-Thought (CoT)

Ask the model to show reasoning steps.

Analyze this claim and determine liability. Think step by step:

Claim: "Vehicle A ran a red light and hit Vehicle B"

Step 1: Identify traffic rule violation

Step 2: Determine which party caused the accident

Step 3: Assess liability percentage

Step 4: Provide recommendation

4. System + User + Assistant Message Pattern

```
messages = [
    {"role": "system", "content": "You are an insurance claims analyst. Analyze claims and provide structured responses. Always output valid JSON."},
    {"role": "user", "content": "Analyze this claim: ..."},
    {"role": "assistant", "content": '{"risk": "HIGH", ...}'},
]
```

Q: How did you implement guardrails for LLM responses?

- Input guardrails: Check for prompt injection, off-topic queries, PII in input
- Output format validation: Parse response as JSON, validate against Pydantic schema
- Content filtering: Azure OpenAI built-in filters for harmful content
- Hallucination check: Verify extracted values exist in source document
- Confidence threshold: If model says "I am not sure", route to human review
- Length validation: Response should not exceed expected length for the task

```
from pydantic import BaseModel, validator
```

```
class ClaimAnalysis(BaseModel):
    risk_level: str # HIGH, MEDIUM, LOW
    liability_pct: float
    reasoning: str

    @validator("risk_level")
    def valid_risk(cls, v):
        assert v in ["HIGH", "MEDIUM", "LOW"]
        return v

    @validator("liability_pct")
    def valid_pct(cls, v):
        assert 0 <= v <= 100
        return v

# Parse LLM output
try:
    result = ClaimAnalysis.parse_raw(llm_response)
```

```
except ValidationError as e:  
    # Fallback: retry with stricter prompt or human review  
    handle_invalid_response(e)
```

Day 5 Practice Tasks

- Write prompts for: zero-shot, few-shot, chain-of-thought for your claims use case
- Implement output validation with Pydantic for LLM responses
- Practice explaining LLM architecture (transformer, attention, pre-training)
- List 5 guardrail techniques and implement 2 of them in code

DAY 6: RAG ARCHITECTURE & VECTOR DATABASES

RAG Architecture (Most Important Topic)

Q: Explain the complete RAG architecture you built.

RAG (Retrieval-Augmented Generation) combines information retrieval with LLM generation to produce grounded, factual responses.

Indexing Pipeline (Offline)

- Step 1 - Document Loading: Azure Document Intelligence extracts text, tables, sections from insurance PDFs
- Step 2 - Chunking: Split documents into smaller pieces (512 tokens, 50-token overlap)
- Step 3 - Embedding: Each chunk is embedded using OpenAI text-embedding-ada-002 (1536 dimensions)
- Step 4 - Storage: Embeddings + metadata + original text stored in PGVector

Retrieval Pipeline (Online)

- Step 1 - Query Embedding: User query is embedded using the same embedding model
- Step 2 - Similarity Search: Cosine similarity search in PGVector, retrieve top-k (5-10) chunks
- Step 3 - Context Assembly: Retrieved chunks + metadata assembled into context
- Step 4 - Prompt Construction: System prompt + context + user query combined
- Step 5 - LLM Generation: Azure OpenAI GPT-4 generates response grounded in context
- Step 6 - Post-processing: Validate output, add citations, format response

Q: What chunking strategies exist and which did you use?

- Fixed-size chunking: Split by character/token count. Simple but may split sentences mid-way.
- Recursive character splitting: Tries to split by paragraphs, then sentences, then words. Preserves structure better. THIS IS WHAT WE USED with LangChain RecursiveCharacterTextSplitter.
- Semantic chunking: Uses embeddings to detect topic boundaries. More accurate but slower.
- Document-structure aware: Uses Document Intelligence layout info to split by sections/headers. Used for insurance policy documents where sections have legal significance.
- Overlap: 10-20% overlap between chunks prevents losing context at boundaries.

```
from langchain.text_splitter import RecursiveCharacterTextSplitter

splitter = RecursiveCharacterTextSplitter(
    chunk_size=512,
    chunk_overlap=50,
    length_function=len,
    separators=["\n\n", "\n", ". ", " ", "", ""])
chunks = splitter.split_documents(documents)
```

Q: What is cosine similarity and why is it used for embeddings?

Cosine similarity measures the angle between two vectors, ignoring magnitude. Range: -1 to 1 (1 = identical direction).

- Formula: $\cos(A,B) = (A \cdot B) / (||A|| * ||B||)$
- Why cosine over Euclidean? Embeddings can have different magnitudes. Cosine focuses on direction (semantic meaning), not scale.
- In PGVector: <=> operator computes cosine distance (1 - cosine similarity)

```
import numpy as np
```

```
def cosine_similarity(a, b):
    return np.dot(a, b) / (np.linalg.norm(a) * np.linalg.norm(b))

# PGVector query
# ORDER BY embedding <=> query_embedding -- cosine distance
# Smaller distance = more similar
```

PGVector Deep Dive

Q: Why did you choose PGVector? What are its limitations?

- Pros: Runs inside PostgreSQL (existing infra), SQL-based filtering + vector search combined, ACID compliance, no separate service to manage, open-source
- Cons: Slower than purpose-built vector DBs at very large scale (100M+ vectors), limited to single-node (no distributed), index needs tuning
- Alternatives: Pinecone (managed, scalable), Weaviate (hybrid search), Milvus (distributed), FAISS (in-memory, fastest)
- Our choice rationale: "We had 50K-100K documents. PGVector handled this scale well, and we avoided adding another service to our architecture."

Q: Explain PGVector indexing strategies.

- IVFFlat: Clusters vectors into lists, searches only nearest clusters. Faster but approximate. Good for medium datasets.
- HNSW (Hierarchical Navigable Small World): Graph-based index. Better recall than IVFFlat. Slower to build but faster queries. Recommended for most use cases.

```
-- IVFFlat index (faster build, lower recall)
CREATE INDEX ON documents
USING ivfflat (embedding vector_cosine_ops)
WITH (lists = 100); -- sqrt(n_rows) is a good starting point

-- HNSW index (slower build, better recall)
CREATE INDEX ON documents
USING hnsw (embedding vector_cosine_ops)
WITH (m = 16, ef_construction = 64);

-- Filtered search (metadata + vector)
SELECT content, 1 - (embedding <=> query_emb) AS similarity
FROM documents
WHERE doc_type = 'policy' AND year = 2024
ORDER BY embedding <=> query_emb
LIMIT 5;
```

Q: How do you evaluate RAG pipeline quality?

- Retrieval metrics: Hit rate (is the answer in top-k?), MRR (Mean Reciprocal Rank), NDCG
- Generation metrics (RAGAS): Faithfulness, answer relevance, context precision, context recall
- End-to-end: Human evaluation on sample queries, A/B testing
- Failure analysis: Log queries where users gave negative feedback, analyze retrieval misses

```
from ragas import evaluate
from ragas.metrics import faithfulness, answer_relevancy,
    context_precision, context_recall

result = evaluate(
    dataset,
    metrics=[faithfulness, answer_relevancy,
             context_precision, context_recall]
```

```
)  
print(result) # scores 0-1 for each metric
```

Day 6 Practice Tasks

- Draw the complete RAG architecture diagram on paper
- Implement a simple RAG pipeline: embed docs, store in PGVector, query
- Compare IVFFlat vs HNSW index performance on sample data
- Write RAGAS evaluation code and interpret the results

DAY 7: LANGCHAIN & LLM APPLICATION DEVELOPMENT

LangChain Core Concepts

Q: What is LangChain and why did you use it?

LangChain is a framework for building LLM-powered applications. It provides abstractions for common patterns like RAG, agents, and chains.

- Why LangChain: Standardized interfaces for different LLMs, built-in PGVector integration, prompt management, chain composition, output parsing
- Key components: Models (LLM wrappers), Prompts (templates), Chains (sequences), Retrievers (search), Memory (conversation history)

Q: Explain the LangChain components you used in your RAG pipeline.

1. Document Loaders

```
from langchain.document_loaders import PyPDFLoader
from langchain.document_loaders import AzureAIDocumentIntelligenceLoader

# Simple PDF loading
loader = PyPDFLoader("policy.pdf")
docs = loader.load()

# Azure Document Intelligence (better for complex docs)
loader = AzureAIDocumentIntelligenceLoader(
    api_endpoint="https://your-resource.cognitiveservices.azure.com/",
    api_key="your-key",
    file_path="policy.pdf",
    api_model="prebuilt-layout"
)
docs = loader.load()
```

2. Text Splitters

```
from langchain.text_splitter import RecursiveCharacterTextSplitter

splitter = RecursiveCharacterTextSplitter(
    chunk_size=512, chunk_overlap=50
)
chunks = splitter.split_documents(docs)
```

3. Embeddings + Vector Store

```
from langchain_openai import AzureOpenAIEmbeddings
from langchain_community.vectorstores import PGVector

embeddings = AzureOpenAIEmbeddings(
    azure_deployment="text-embedding-ada-002",
    azure_endpoint="https://your-resource.openai.azure.com/"
)

vectorstore = PGVector.from_documents(
    documents=chunks,
    embedding=embeddings,
    connection_string="postgresql://user:pass@host/db",
    collection_name="insurance_docs"
```

)

4. Retriever + Chain

```

from langchain_openai import AzureChatOpenAI
from langchain.chains import RetrievalQA
from langchain.prompts import ChatPromptTemplate

llm = AzureChatOpenAI(
    azure_deployment="gpt-4",
    temperature=0
)

retriever = vectorstore.as_retriever(
    search_type="similarity",
    search_kwargs={"k": 5}
)

prompt = ChatPromptTemplate.from_messages([
    ("system", """"You are an insurance claims analyst.
    Use only the provided context to answer.
    If the answer is not in context, say "I don't know".
    Context: {context}"""),
    ("human", "{question}")
])

chain = RetrievalQA.from_chain_type(
    llm=llm,
    retriever=retriever,
    chain_type_kwargs={"prompt": prompt},
    return_source_documents=True
)

result = chain.invoke({"query": "What is the liability policy?"})
print(result["result"])
print(result["source_documents"])

```

Q: What is LCEL (LangChain Expression Language)?

LCEL is the modern way to compose chains in LangChain using the pipe (|) operator. More readable and supports streaming.

```

from langchain_core.runnables import RunnablePassthrough
from langchain_core.output_parsers import StrOutputParser

# Modern LCEL chain
chain = (
    {"context": retriever, "question": RunnablePassthrough()}
    | prompt
    | llm
    | StrOutputParser()
)

response = chain.invoke("What is the coverage limit?")

# With streaming
for chunk in chain.stream("What is the coverage limit?"):
    print(chunk, end="", flush=True)

```

Q: What are the different chain types for RAG?

- stuff: Puts all retrieved docs in one prompt. Best when total context fits in window. Simplest and most common.
- map_reduce: Processes each doc independently, then combines. Good for many/large docs. More LLM calls = higher cost.
- refine: Iteratively refines answer with each doc. Better quality but sequential = slow.
- map_rerank: Each doc gets scored, best answer selected. Good when answer is likely in one doc.

In our project: "We used stuff for single policy queries (context fits easily) and map_reduce for cross-document comparison tasks."

Output Parsing & Structured Output

Q: How do you get structured output from LLMs?

```
from langchain_core.output_parsers import PydanticOutputParser
from pydantic import BaseModel, Field

class LiabilityDecision(BaseModel):
    liability_party: str = Field(description="Party at fault")
    liability_pct: float = Field(description="Liability percentage")
    reasoning: str = Field(description="Step by step reasoning")
    confidence: str = Field(description="HIGH, MEDIUM, or LOW")

parser = PydanticOutputParser(pydantic_object=LiabilityDecision)

prompt = ChatPromptTemplate.from_messages([
    ("system", """"Analyze the accident and determine liability.
    {format_instructions}"""),
    ("human", "{accident_description}")
])

chain = prompt | llm | parser

result = chain.invoke({
    "accident_description": "Vehicle A ran red light...",
    "format_instructions": parser.get_format_instructions()
})
# result is a LiabilityDecision object
print(result.liability_pct) # 80.0
```

Day 7 Practice Tasks

- Build a complete RAG chain using LCEL with PGVector
- Implement structured output parsing with Pydantic
- Compare stuff vs map_reduce chain types on sample docs
- Add conversation memory to your RAG chain

DAY 8: AZURE AI SERVICES

Azure OpenAI Service

Q: How does Azure OpenAI differ from OpenAI directly?

- Data privacy: Your data is NOT used for model training. Enterprise data stays in your Azure tenant.
- Compliance: SOC2, HIPAA, GDPR compliant. Critical for insurance industry.
- Content filtering: Built-in content moderation (can be configured or removed)
- Networking: Private endpoints, VNET integration. No data leaves your network.
- SLA: 99.9% uptime guarantee with enterprise support
- Same models: GPT-4, GPT-4o, text-embedding-ada-002, etc.

Q: Explain Azure OpenAI deployment and rate limits.

```
from openai import AzureOpenAI

client = AzureOpenAI(
    api_key="your-key",
    api_version="2024-06-01",
    azure_endpoint="https://your-resource.openai.azure.com/"
)

response = client.chat.completions.create(
    model="gpt-4", # This is your DEPLOYMENT name
    messages=[
        {"role": "system", "content": "You are an assistant"},
        {"role": "user", "content": "Hello"}
    ],
    temperature=0,
    max_tokens=500
)
```

- Rate limits: Tokens Per Minute (TPM) and Requests Per Minute (RPM)
- Handling rate limits: Implement exponential backoff, use multiple deployments, queue requests
- PTU (Provisioned Throughput Units): Reserved capacity for production workloads with guaranteed throughput

Azure Document Intelligence

Q: Explain Azure Document Intelligence and its models.

Cloud-based AI service that extracts text, structure, and data from documents.

- Prebuilt models: Invoice, Receipt, ID document, Business card, W2 tax form
- Layout model: Extracts text, tables, selection marks, structure from any document. This is what we used most.
- Custom models: Train on your own document types with labeled data
- Composed models: Combine multiple custom models for documents with varying formats

```
from azure.ai.documentintelligence import DocumentIntelligenceClient
from azure.core.credentials import AzureKeyCredential

client = DocumentIntelligenceClient(
    endpoint="https://your-resource.cognitiveservices.azure.com/",
    credential=AzureKeyCredential("your-key")
)
```

```
# Analyze document with Layout model
with open("policy.pdf", "rb") as f:
    poller = client.begin_analyze_document(
        "prebuilt-layout", body=f
    )
result = poller.result()

# Extract text by pages
for page in result.pages:
    for line in page.lines:
        print(line.content)

# Extract tables
for table in result.tables:
    for cell in table.cells:
        print(f"Row {cell.row_index}, Col {cell.column_index}: "
              f"{cell.content}")
```

Azure AI Foundry

Q: What is Azure AI Foundry?

Azure AI Foundry (formerly Azure AI Studio) is a unified platform for building, evaluating, and deploying AI applications.

- Model catalog: Access to Azure OpenAI, Meta LLaMA, Mistral, Cohere and other models
- Prompt flow: Visual tool for building LLM workflows (chains)
- Evaluation: Built-in metrics for evaluating LLM applications (groundedness, relevance, coherence)
- Fine-tuning: Fine-tune models on your data within the platform
- Content safety: Configure and test content filters
- Deployment: One-click deployment to Azure endpoints

Day 8 Practice Tasks

- Set up Azure OpenAI and make API calls with different parameters
- Use Document Intelligence to extract data from a sample PDF
- Practice explaining Azure vs OpenAI differences for enterprise use
- Explore Azure AI Foundry evaluation features

DAY 9: MLOPS - DOCKER, KUBERNETES, CI/CD & FAST

FastAPI for Model Serving

Q: How did you build your FastAPI service for model serving?

```
from fastapi import FastAPI, HTTPException
from pydantic import BaseModel
import joblib
import numpy as np

app = FastAPI(title="Claims Prediction API")

# Load model at startup (not per request)
model = joblib.load("xgboost_model.pkl")

class ClaimRequest(BaseModel):
    claim_type: str
    accident_description: str
    policy_age_days: int
    prior_claims: int

class ClaimResponse(BaseModel):
    predicted_amount: float
    risk_level: str
    confidence: float

@app.post("/predict", response_model=ClaimResponse)
async def predict(request: ClaimRequest):
    try:
        features = preprocess(request)
        prediction = model.predict(features)[0]
        confidence = model.predict_proba(features).max()
        return ClaimResponse(
            predicted_amount=round(prediction, 2),
            risk_level=categorize_risk(prediction),
            confidence=round(confidence, 3)
        )
    except Exception as e:
        raise HTTPException(status_code=500, detail=str(e))

@app.get("/health")
async def health():
    return {"status": "healthy", "model_version": "v2.1"}
```

Q: How do you handle async calls for LLM endpoints?

```
import asyncio
from openai import AsyncAzureOpenAI

client = AsyncAzureOpenAI(...)

@app.post("/analyze")
async def analyze_claim(request: ClaimRequest):
    # Non-blocking LLM call
```

```

response = await client.chat.completions.create(
    model="gpt-4",
    messages=[...],
    temperature=0
)
return {"analysis": response.choices[0].message.content}

# Multiple concurrent LLM calls
@app.post("/batch-analyze")
async def batch_analyze(requests: list[ClaimRequest]):
    tasks = [analyze_single(r) for r in requests]
    results = await asyncio.gather(*tasks)
    return results

```

Docker

Q: Write a Dockerfile for your ML/LLM service.

```

# Multi-stage build for smaller image
FROM python:3.11-slim as builder
WORKDIR /app
COPY requirements.txt .
RUN pip install --no-cache-dir -r requirements.txt

FROM python:3.11-slim
WORKDIR /app
COPY --from=builder /usr/local/lib/python3.11 /usr/local/lib/python3.11
COPY --from=builder /usr/local/bin /usr/local/bin
COPY . .

# Non-root user for security
RUN useradd -m appuser
USER appuser

EXPOSE 8000
HEALTHCHECK CMD curl -f http://localhost:8000/health || exit 1
CMD ["uvicorn", "main:app", "--host", "0.0.0.0", "--port", "8000"]

```

Q: What are Docker best practices for ML services?

- Use multi-stage builds to reduce image size
- Use .dockerignore to exclude data files, notebooks, .git
- Pin dependency versions in requirements.txt
- Run as non-root user
- Add health checks
- Use slim/alpine base images
- Layer ordering: dependencies first (cached), code last (changes often)

Kubernetes

Q: Explain your Kubernetes deployment architecture.

```

# deployment.yaml
apiVersion: apps/v1
kind: Deployment
metadata:
  name: claims-predictor

```

```

spec:
  replicas: 3
  selector:
    matchLabels:
      app: claims-predictor
  template:
    metadata:
      labels:
        app: claims-predictor
    spec:
      containers:
        - name: api
          image: registry.azurecr.io/claims-predictor:v2.1
          ports:
            - containerPort: 8000
          resources:
            requests:
              memory: "512Mi"
              cpu: "250m"
            limits:
              memory: "1Gi"
              cpu: "500m"
          livenessProbe:
            httpGet:
              path: /health
              port: 8000
            initialDelaySeconds: 30
            periodSeconds: 10
          readinessProbe:
            httpGet:
              path: /health
              port: 8000
            initialDelaySeconds: 5
      ---
    apiVersion: v1
    kind: Service
    metadata:
      name: claims-predictor-svc
    spec:
      selector:
        app: claims-predictor
      ports:
        - port: 80
          targetPort: 8000
      type: ClusterIP

```

Q: What are Helm charts and why did you use them?

Helm is a package manager for Kubernetes. Helm charts are templated K8s manifests.

- Templates: Use variables ({{ .Values.image.tag }}) instead of hardcoded values
- Values files: Different configs per environment (dev.yaml, staging.yaml, prod.yaml)
- Versioning: Track deployment versions, easy rollback with helm rollback
- Reusability: Same chart for multiple microservices with different values

CI/CD with GitHub Actions

Q: Explain your CI/CD pipeline.

```
# .github/workflows/deploy.yml
name: ML Pipeline CI/CD
on:
  push:
    branches: [main]
  pull_request:
    branches: [main]

jobs:
  test:
    runs-on: ubuntu-latest
    steps:
      - uses: actions/checkout@v4
      - uses: actions/setup-python@v5
        with:
          python-version: "3.11"
      - run: pip install -r requirements.txt
      - run: pytest tests/ -v --cov=src

  build-deploy:
    needs: test
    if: github.ref == 'refs/heads/main'
    runs-on: ubuntu-latest
    steps:
      - uses: actions/checkout@v4
      - name: Build Docker image
        run: docker build -t claims-predictor:$GITHUB_SHA .
      - name: Push to ACR
        run: |
          docker tag claims-predictor:$GITHUB_SHA \
            myregistry.azurecr.io/claims-predictor:$GITHUB_SHA
          docker push myregistry.azurecr.io/claims-predictor:$GITHUB_SHA
      - name: Deploy to K8s
        run: |
          helm upgrade --install claims-predictor ./helm \
            --set image.tag=$GITHUB_SHA
```

DVC & MLflow**Q: How do DVC and MLflow work together in your pipeline?**

- DVC (Data Version Control): Tracks large data files and model artifacts outside git. Stores in Azure Data Lake. Links data versions to git commits.
- MLflow Tracking: Logs hyperparameters, metrics, and artifacts for each experiment run. Compare runs side by side.
- MLflow Model Registry: Promote models through stages (Staging -> Production). Version control for models.

```
# DVC
dvc init
dvc add data/claims.csv      # tracks data file
dvc remote add -d azure azure://bucket/dvc-store
dvc push                     # uploads to Azure Data Lake

# MLflow experiment tracking
import mlflow
```

```
mlflow.set_experiment("claims-prediction")
with mlflow.start_run():
    mlflow.log_params({"lr": 0.1, "max_depth": 6})
    model.fit(X_train, y_train)
    score = model.score(X_test, y_test)
    mlflow.log_metric("f1_score", score)
    mlflow.sklearn.log_model(model, "model")
```

Day 9 Practice Tasks

- Build a FastAPI app that serves an ML model, then Dockerize it
- Write a Kubernetes deployment manifest with health checks
- Create a GitHub Actions workflow that runs tests and builds Docker
- Practice explaining your CI/CD pipeline end-to-end

DAY 10: SYSTEM DESIGN, MODEL MONITORING & MOC

Model Monitoring (Evidently AI + Grafana)

Q: How did you monitor ML models in production?

We implemented a comprehensive monitoring stack using Evidently AI for drift detection and Grafana for visualization.

- Data Drift: Monitor if input feature distributions change over time. Detected using PSI (Population Stability Index) and KS test (Kolmogorov-Smirnov).
- Concept Drift: Monitor if the relationship between features and target changes. Detected by tracking model accuracy/F1 over time.
- Performance Metrics: Accuracy, F1, precision, recall tracked daily/weekly
- Operational Metrics: Prediction latency (p50, p95, p99), error rate, throughput
- LLM-specific: Token usage, cost per query, hallucination rate, guardrail trigger rate

Q: Explain your Grafana dashboard setup.

```
-- SQL query for Grafana panel: Daily prediction distribution
SELECT
    DATE(prediction_time) AS day,
    AVG(predicted_amount) AS avg_prediction,
    PERCENTILE_CONT(0.95) WITHIN GROUP
        (ORDER BY predicted_amount) AS p95_prediction,
    COUNT(*) AS total_predictions
FROM predictions
WHERE prediction_time > NOW() - INTERVAL '30 days'
GROUP BY DATE(prediction_time)
ORDER BY day;

-- LLM monitoring: Token usage and latency
SELECT
    DATE(request_time) AS day,
    AVG(total_tokens) AS avg_tokens,
    AVG(latency_ms) AS avg_latency,
    SUM(total_tokens) * 0.00003 AS estimated_cost_usd
FROM llm_requests
GROUP BY DATE(request_time);
```

Q: What is MCP and how did you use it with Claude Desktop for monitoring?

MCP (Model Context Protocol) allows Claude Desktop to interact with external tools and data sources. We built an MCP server that connects Claude to our monitoring data.

- Claude Desktop queries Evidently reports via MCP
- Generates natural-language summaries: "Feature X has drifted 15% from baseline. This correlates with increased prediction errors in the Northeast region."
- Proactive alerting: Claude identifies anomalies and suggests investigation paths

System Design Questions

Q: Design a RAG system that handles 10,000+ documents for an insurance company.

Requirements

- 10,000+ policy documents, claims, regulations
- Sub-second retrieval, accurate LLM responses
- Multi-user, concurrent access
- Data security and compliance

Architecture

1. Ingestion Layer: Azure Document Intelligence -> Chunking (RecursiveCharacterTextSplitter, 512 tokens, 50 overlap) -> OpenAI Embeddings -> PGVector storage
2. API Layer: FastAPI with async endpoints. Load balancer -> multiple replicas on K8s.
3. Retrieval Layer: PGVector with HNSW index. Hybrid search: vector similarity + metadata filters (doc_type, date, policy_number).
4. Generation Layer: Azure OpenAI GPT-4. Prompt template with system role + retrieved context + user query. Temperature=0 for factual responses.
5. Monitoring: Evidently AI for drift, Grafana dashboards, MCP + Claude for insights.
6. Security: Azure VNET, private endpoints, role-based access, PII filtering in prompts.

Scaling Considerations

- Horizontal scaling: K8s replicas for API layer
- Caching: Redis cache for frequent queries
- Async processing: Background ingestion pipeline
- Index optimization: HNSW index tuning, partition by document type

Behavioral / Project Questions

Q: Tell me about your most impactful project.

Sample answer: "At Allianz, I built a RAG-based liability prediction system that analyzes accident descriptions and recommends liability decisions. Previously, claim handlers manually reviewed each case, which took 30-45 minutes. Our system retrieves similar past cases from a PGVector database, provides the LLM with relevant precedents, and generates a structured recommendation with reasoning. This improved decision consistency by 30% and significantly reduced review time. The model saved over 5 million GBP in the first year through more accurate claim predictions."

Q: How do you handle a situation where the model is not performing well?

Structured approach:

- Step 1: Identify the problem - Is it data drift, concept drift, or a bug? Check Grafana dashboards and Evidently reports.
- Step 2: Root cause analysis - Analyze misclassified samples. Are they from a specific segment? Check feature distributions.
- Step 3: Fix - If data drift: retrain on recent data. If concept drift: re-engineer features, consider new model. If bug: fix pipeline.
- Step 4: Validate - A/B test new model vs old. Compare metrics on held-out set.
- Step 5: Deploy - Blue-green deployment. Monitor closely for 1-2 weeks.
- Step 6: Prevent - Add monitoring alerts for earlier detection next time.

Q: How do you ensure responsible AI in insurance?

- Bias detection: Check model predictions across demographics. Use fairness metrics.
- Explainability: SHAP values for ML models, source citations for RAG responses.
- Human-in-the-loop: LLM recommendations flagged for human review, not auto-approved.
- Content filtering: Azure OpenAI built-in filters + custom guardrails.

- Data privacy: PII masking in prompts, data stays in Azure tenant, no training on our data.

Quick Fire Technical Questions

Q: Difference between fine-tuning and RAG?

- Fine-tuning: Modifies model weights with your data. Good for style/format. Expensive, needs retraining for new data.
- RAG: Retrieves relevant data at query time. Good for factual/current info. No retraining needed, just update the vector store.
- In our case: "RAG was the right choice because insurance policies change frequently. Fine-tuning would require retraining every time a policy updates."

Q: What is hallucination and how do you prevent it?

- Hallucination: LLM generates confident but incorrect information not present in the source.
- Prevention: Grounding with RAG, low temperature, explicit "only use provided context" instructions, output validation, confidence thresholds, source citations.

Q: Explain microservices vs monolith for ML systems.

- Monolith: Single service handles ingestion, prediction, monitoring. Simpler to deploy but harder to scale independently.
- Microservices: Separate services for ingestion, prediction, monitoring. Can scale and deploy independently. Our approach at Allianz.

Q: What is A/B testing for ML models?

- Route a percentage of traffic to the new model (canary deployment)
- Compare metrics (accuracy, latency, user satisfaction) between old and new
- Gradually increase traffic to new model if metrics are better
- Rollback instantly if degradation detected

Day 10 Checklist - Final Review

- Can you draw the complete RAG architecture from memory?
- Can you explain each component of your ML pipeline end-to-end?
- Can you write LangChain code for a RAG chain without looking?
- Can you explain XGBoost vs Random Forest with pros/cons?
- Can you write a FastAPI endpoint, Dockerfile, and K8s manifest?
- Can you explain your monitoring setup (Evidently + Grafana + MCP)?
- Do you have a strong "tell me about your project" story ready?
- Can you handle "what would you do differently" questions?
- Practice answering 5 questions out loud (not just reading)
- Get a good night sleep - you are ready!

Interview Tip: Remember: Interviewers value depth over breadth. It is better to explain one project deeply with architecture, trade-offs, and metrics than to list 10 technologies superficially. Always tie your answers back to your insurance domain experience at Allianz.